Course Name:	**Competitive Programming Lab**	**Semester:**	**IV**
Date of Performance:	**20/ 04/ 2026**	**DIV/ Batch No:**	**B2**
Student Name:	**Ashwera Hasan**	**Roll No:**	**16010124107**

Experiment No: 8

Title: Game Theory

Aim of the Experiment: To understand and use game theory to solve the given problem

Objective of the Experiment:
To model and solve application-driven computational problems using game theory.

COs to be achieved:

CO4: Apply matrices, probability, game theory , and string algorithms in competitive programming.

Theory:

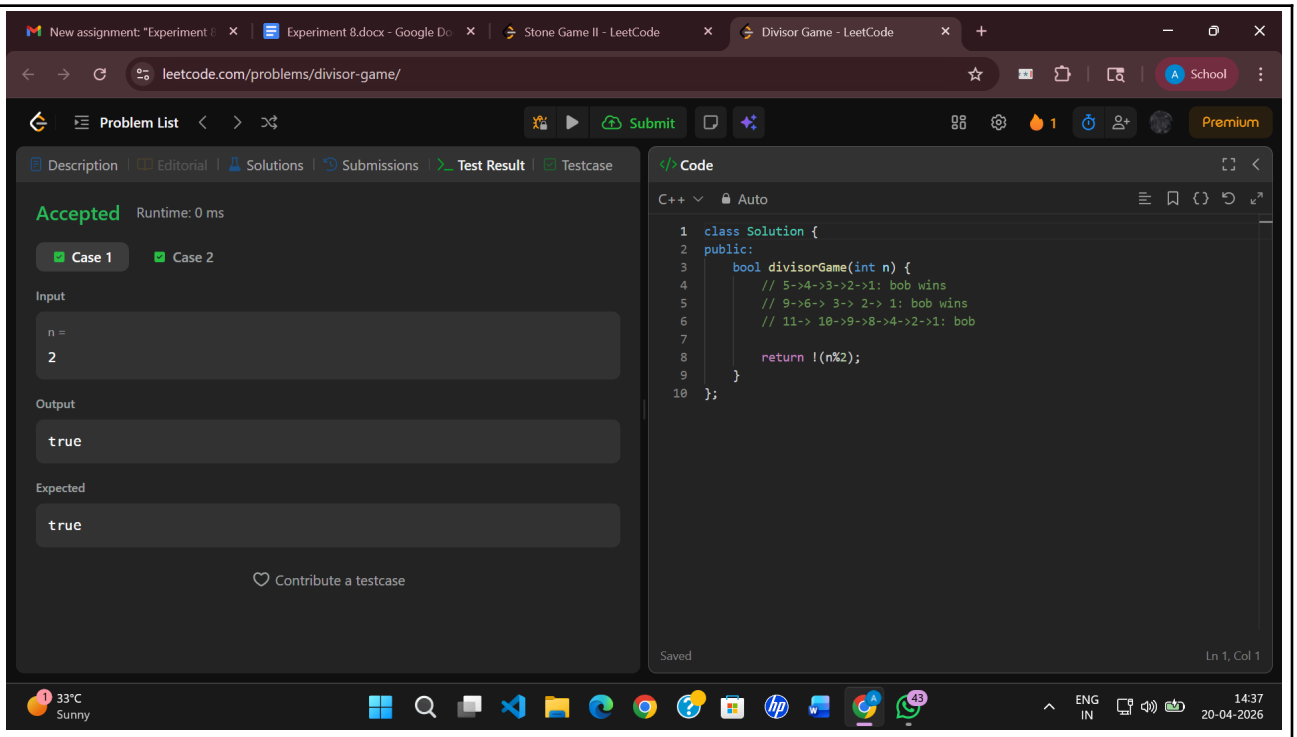
1. Game theory studies decision making in competitive situations where each player's best move depends on the opponent's move.
2. In competitive programming, many game problems are two-player, turn-based, deterministic, and perfect-information; therefore each configuration can be treated as a state.
3. From a state, all legal moves generate child states and together form a game tree. Terminal states are positions where no further move is possible or the result is already known.
4. Minimax assumes optimal play by both sides: MAX chooses the move with the highest value, while MIN chooses the move with the lowest value. The state value may be win/loss, score difference, or another utility.
5. When the same state appears again, memoization stores the answer and avoids repeated computation. Alpha-beta pruning further skips branches that cannot affect the final minimax answer.

Problem Statements:

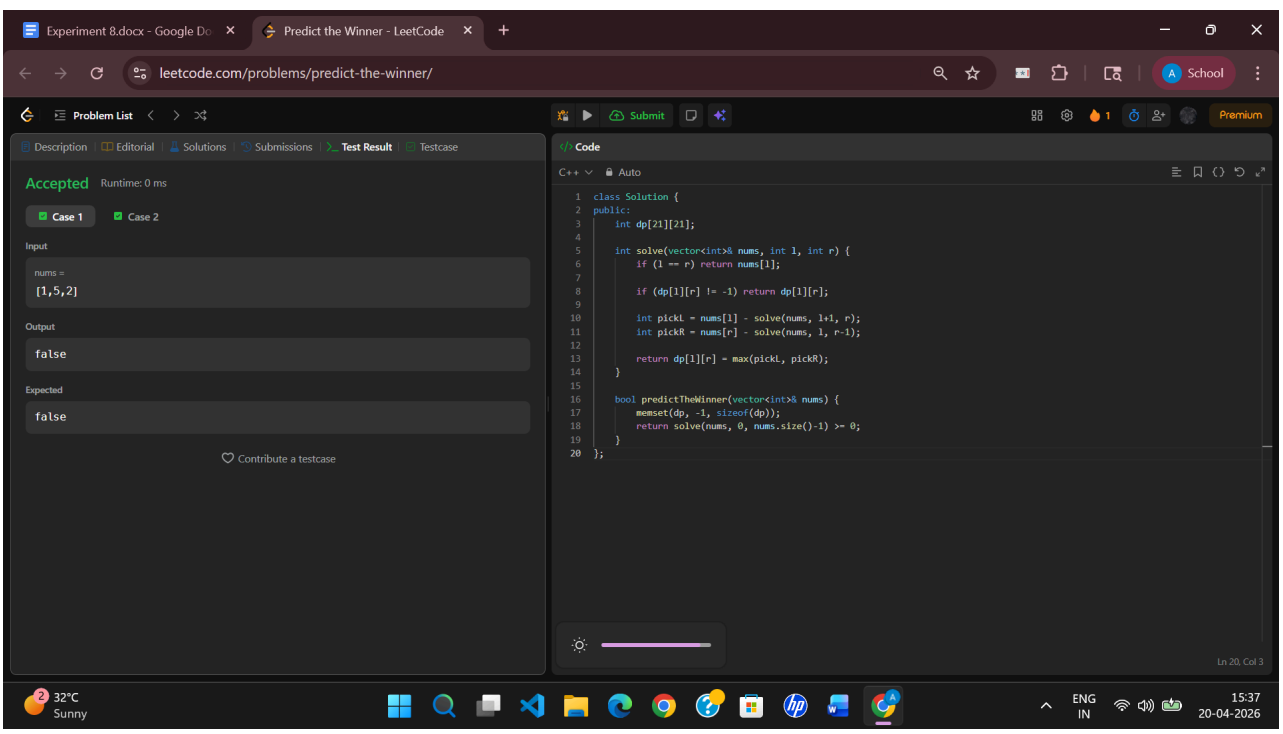
Difficulty (as per lab)	Problem	LeetCode mapping
Easy	Matchstick game	Divisor Game - LeetCode
Medium	Tic Tac Toe	Predict the Winner - LeetCode OR https://leetcode.com/problems/stone-game-ii/description/
Difficult	Nim game	Stone Game III - LeetCode OR Can I Win - LeetCode

Code :

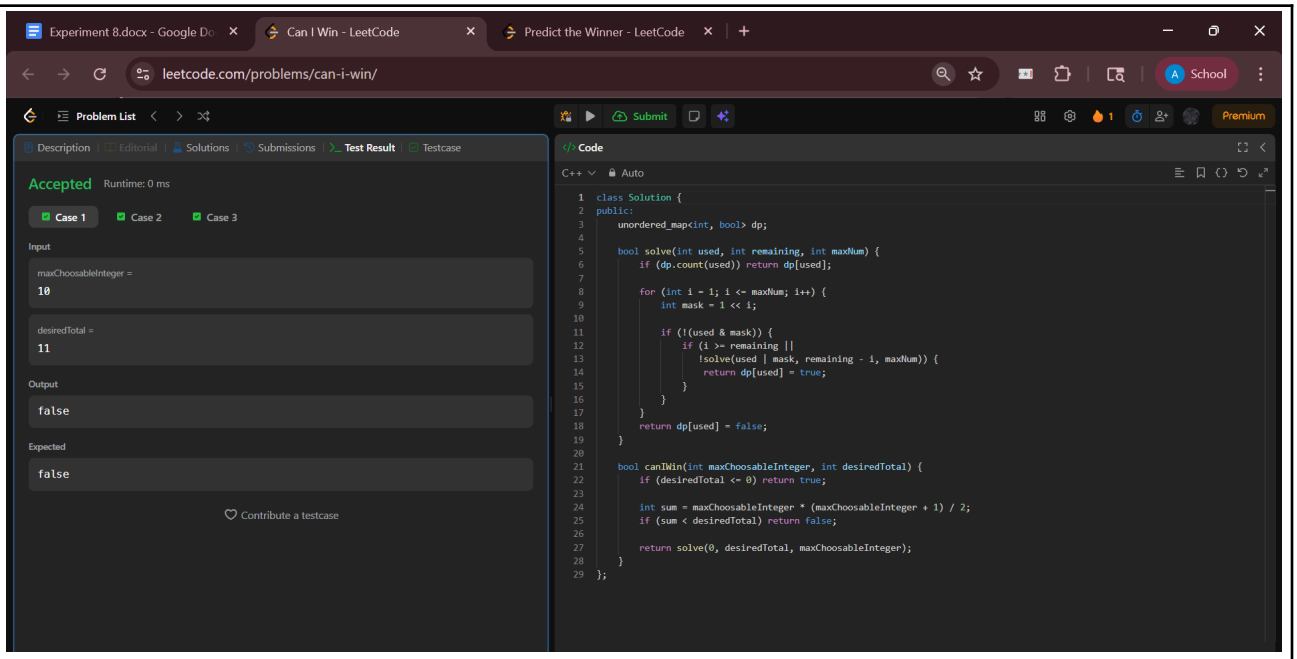
Easy


Divisor Game - LeetCode
leetcode.com/problems/divisor-game/
Accepted
Runtime: 0 ms
Case 1
Case 2
Input
n =
2
Output
true
Expected
true
class Solution {
public:
bool divisorGame(int n) {
// 5->4->3->2->1: bob wins
// 9->6-> 3-> 2-> 1: bob wins
// 11-> 10->9->8->4->2->1: bob
return !(n%2);
}
};

Medium

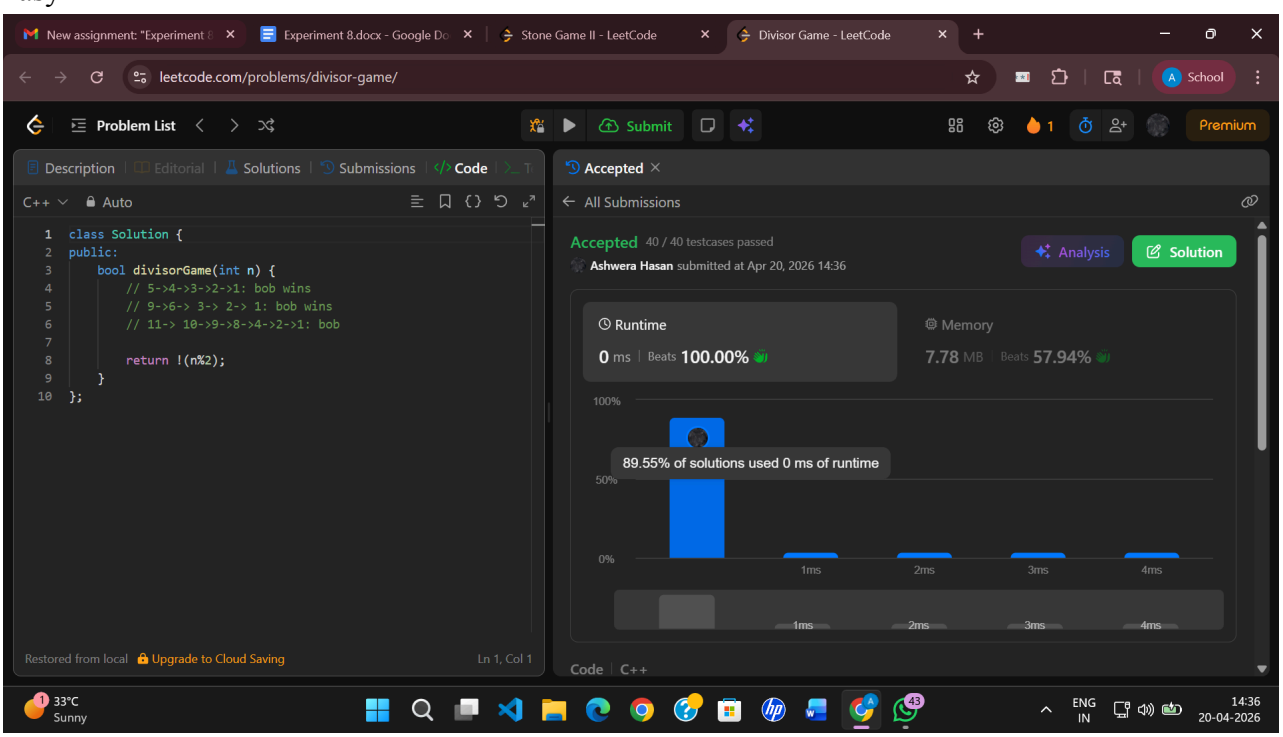

Predict the Winner - LeetCode
leetcode.com/problems/predict-the-winner/
Accepted
Runtime: 0 ms
Case 1
Case 2
Input
nums =
[1,5,2]
Output
false
Expected
false
class Solution {
public:
int dp[21][21];
int solve(vector<int>& nums, int l, int r) {
if (l == r) return nums[l];
if (dp[l][r] != -1) return dp[l][r];
int pickL = nums[l] - solve(nums, l+1, r);
int pickR = nums[r] - solve(nums, l, r-1);
return dp[l][r] = max(pickL, pickR);
}
bool predictTheWinner(vector<int>& nums) {
memset(dp, -1, sizeof(dp));
return solve(nums, 0, nums.size()-1) >= 0;
}
};

Hard

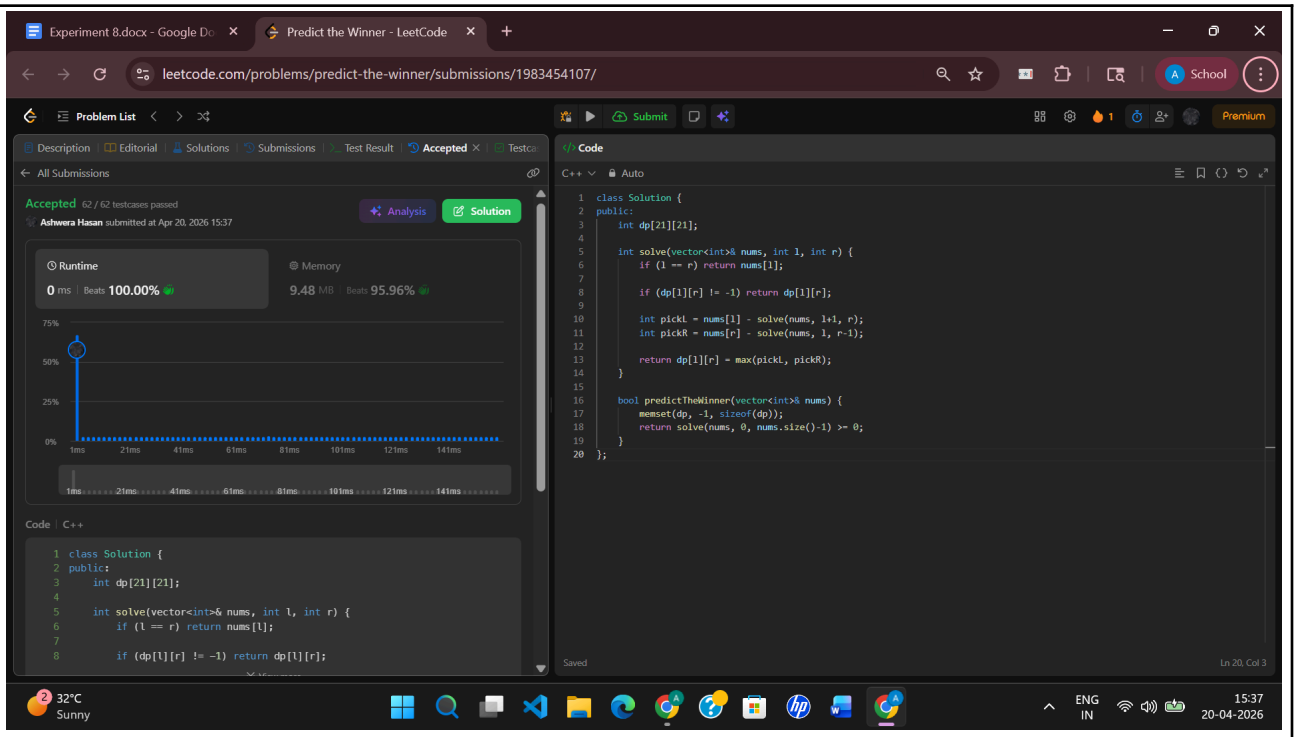


Output:

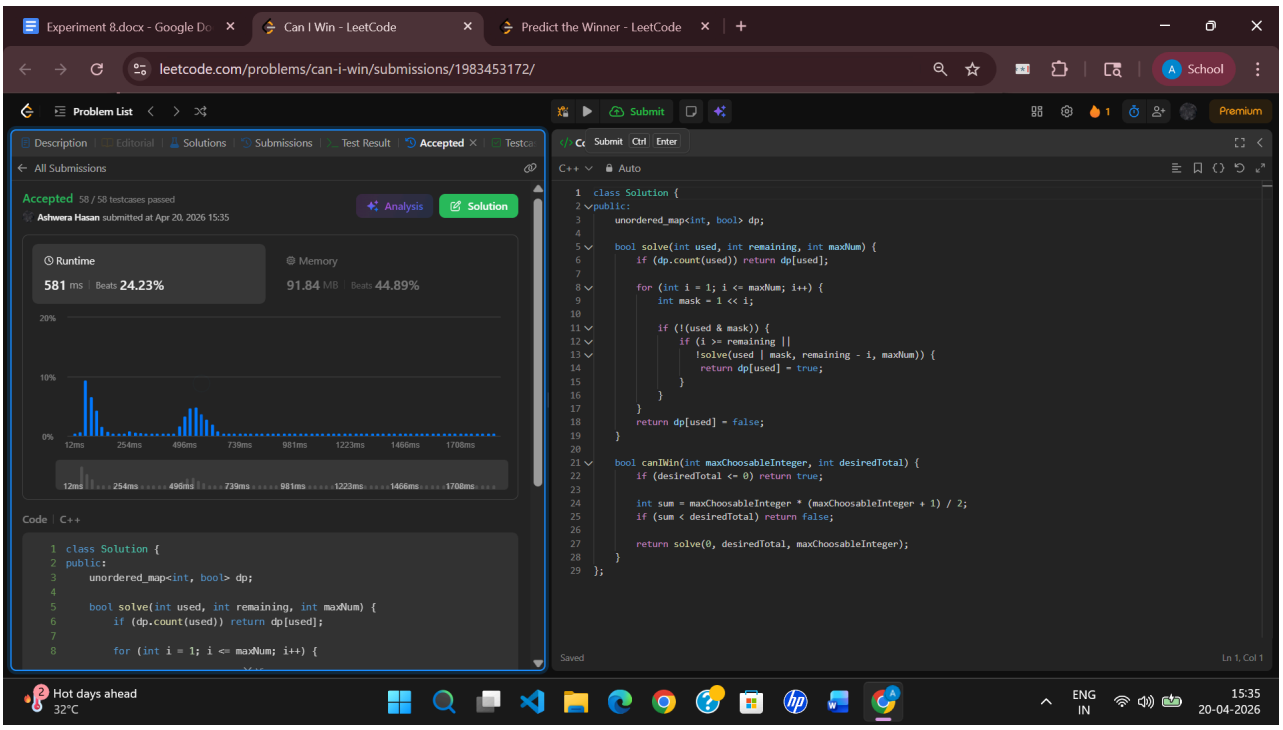
Easy



Medium



Hard



Post Lab Subjective/Objective type Questions:

1. What is the difference between a terminal state, a utility function, and a heuristic evaluation function in a game tree?

Terminal State:
A terminal state is a game position where the game ends. No further moves are possible. The result (win, loss, or draw) is known at this state.

Utility Function:
A utility function assigns a numerical value to terminal states. It gives the exact outcome of the game. For example, +1 for a win, 0 for a draw, and −1 for a loss.

Heuristic Evaluation Function:
A heuristic evaluation function estimates the value of a non-terminal state. It is used when the game tree is too large to search completely. It provides an approximate measure of how favorable a position is.

Difference:
Terminal state represents the end of the game. Utility function gives the exact value of terminal states. Heuristic function gives an estimated value for non-terminal states.

2. Why does alpha-beta pruning not change the final answer produced by minimax, and in which situations does memoization help significantly?

Alpha-Beta Pruning:
Alpha-beta pruning removes branches that cannot affect the final decision. It only skips unnecessary nodes. It does not change the order of evaluation for optimal moves. Hence, the final answer remains the same as minimax.

Reason:
Pruned nodes are never chosen by minimax. They are guaranteed to be worse than already explored options. So ignoring them does not affect the result.

Memoization:
Memoization stores results of already evaluated states. If the same state appears again, the stored result is reused instead of recomputing.

When it helps:
It helps when the game has repeated states. This happens in games with cycles or transpositions. It significantly reduces computation time by avoiding duplicate work.

Conclusion:

Game theory is a mathematical approach used to analyze decision-making in competitive and strategic situations. It helps in determining optimal strategies for players involved in a game. Algorithms like minimax ensure the best possible outcome, while techniques such as alpha-beta pruning improve efficiency. Heuristic functions allow handling of complex problems where full computation is not feasible. Overall, game theory plays an important role in artificial intelligence and real-world problem solving.

Signature of faculty in-charge with Date: